

## ADDITIONAL FINDINGS: LFI & CROSS-SITE SCRIPTING (XSS)

The following section documents additional vulnerability findings captured during the internship week, including a Local File Inclusion (LFI) exploit and multiple Reflected Cross-Site Scripting (XSS) attacks across Low, Medium, and High security levels on DWWA.

### Finding 1: Local File Inclusion (LFI) – File Path Traversal

Using Burp Suite Repeater, a file path traversal (LFI) vulnerability was exploited on PortSwigger Web Security Academy's 'File path traversal, simple case' lab. The filename parameter in the image request was manipulated using directory traversal sequences (`../../../../etc/passwd`) to retrieve the server's `/etc/passwd` file. The response returned the complete Linux user database including system accounts (`root`, `daemon`, `bin`, `sys`, `sync`, `games`, etc.), confirming unauthenticated file read access. The lab was solved successfully as indicated by the 'Congratulations' banner.

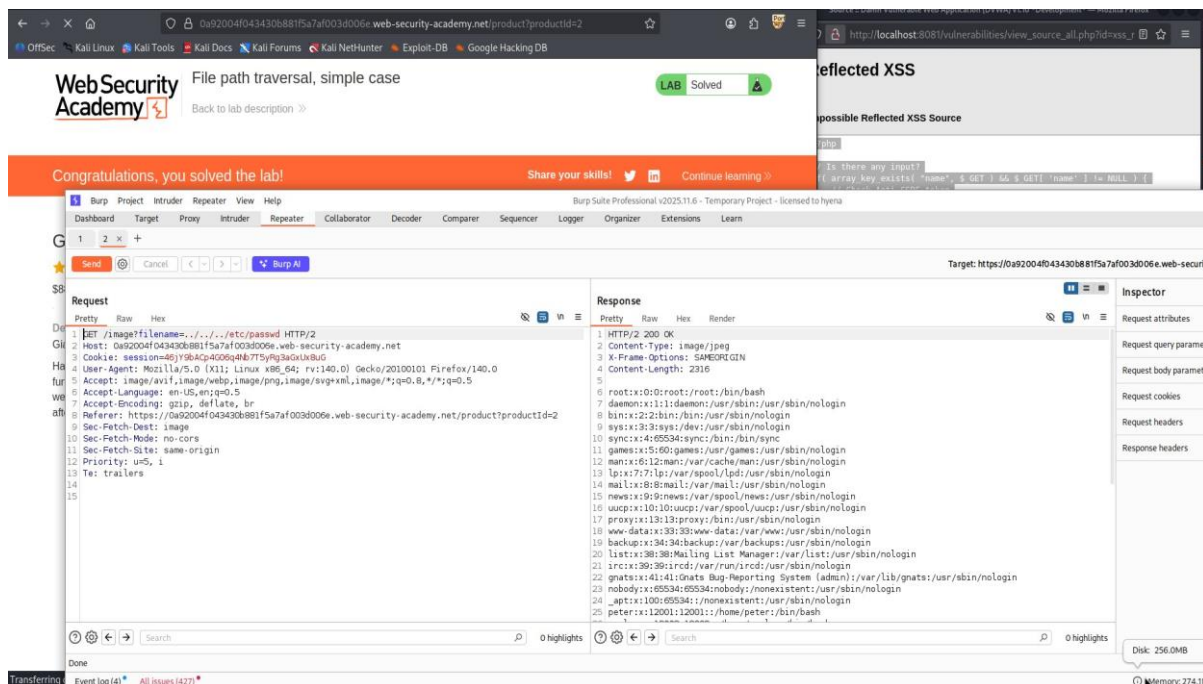


Figure – LFI / File Path Traversal: Burp Suite Repeater showing GET `/image?filename=../../../../etc/passwd` and the full `/etc/passwd` contents returned in the HTTP response. The PortSwigger lab confirms 'Solved' status.

### Finding 2: Reflected XSS – Low Security (Script Tag)

At DWWA's Low security level, no input sanitization is applied. The classic payload `<script>alert(document.cookie)</script>` was injected into the name parameter. The server reflected this input directly into the HTML response, executing JavaScript in the browser context. The alert dialog displayed the full session cookie including `PHPSESSID` and `security=low`, demonstrating successful cookie theft via Reflected XSS. The Low Reflected XSS Source code (visible on the right panel) confirms the input is concatenated directly into the echo statement without any filtering.

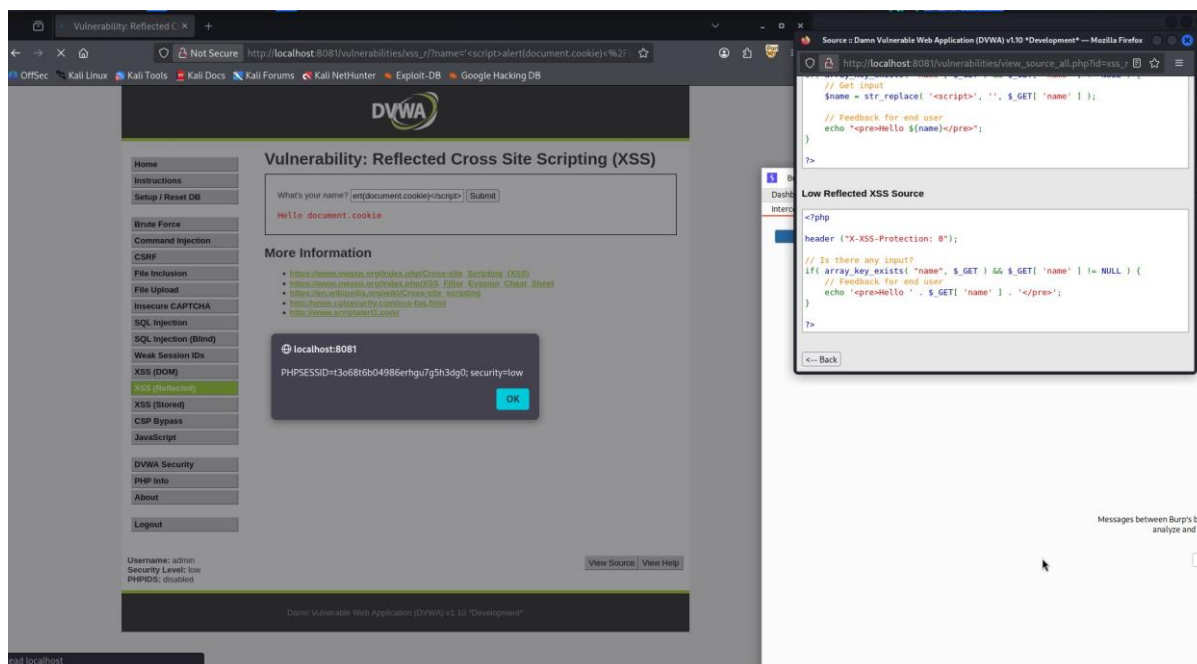


Figure – XSS Low Level: DVWA Reflected XSS (Low) with `<script>alert(document.cookie)</script>` payload. Alert box displays PHPSESSID and security=low cookie values. Source code confirms no sanitization applied.

### Finding 3: Reflected XSS – Medium Security (img Tag Bypass)

At Medium security level, DVWA strips `<script>` tags using `str_replace()`. This was bypassed by using an `<img>` tag with an onerror event handler: `<img src=x onerror=alert(document.cookie)>`. When the browser attempts to load the invalid image source, the onerror event fires, executing the JavaScript payload. The alert dialog displayed the PHPSESSID and security=medium values, confirming the bypass. The Medium Reflected XSS Source code panel on the right confirms only `<script>` tags are filtered, leaving event handlers unblocked. The URL-encoded payload is visible in the browser address bar.

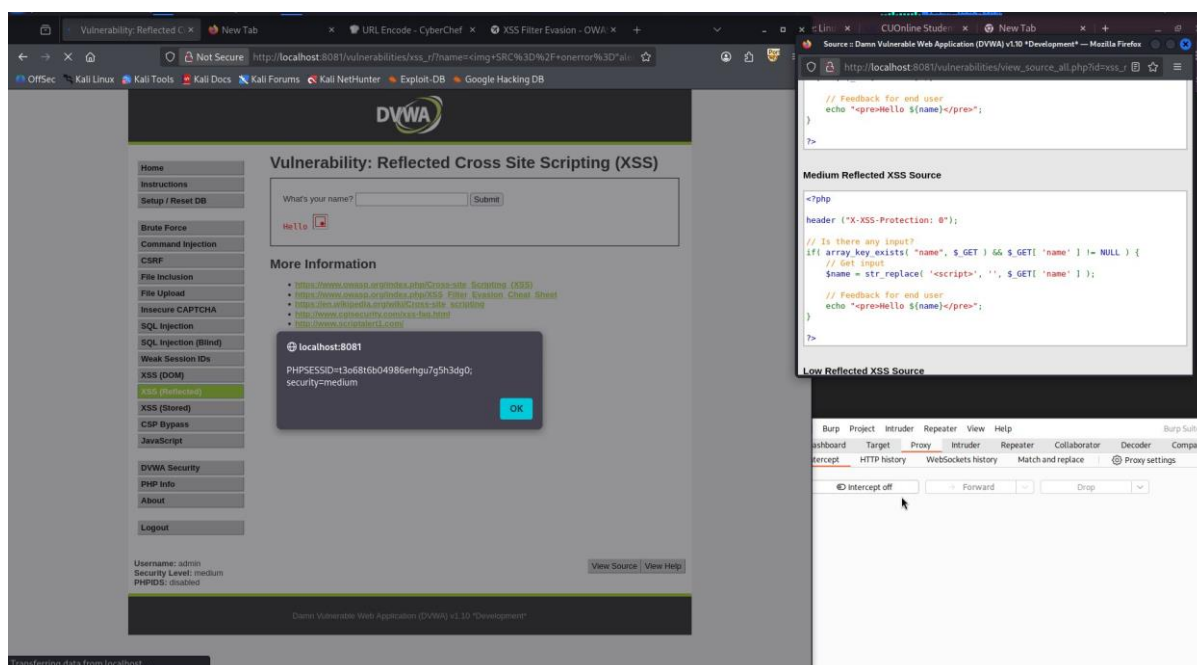


Figure – XSS Medium Level: DVWA Reflected XSS (Medium) bypassed using `<img src=x onerror=alert(document.cookie)>`. Alert shows PHPSESSID and security=medium. Source code confirms only script tags are stripped.

## Finding 4: Reflected XSS – High Security (Advanced Filter Bypass via URL Encoding)

At High security level, DVWA applies a regex-based filter using `preg_replace` to strip a wide range of HTML tags and event handlers. The payload was crafted to bypass this filter by leveraging an `<img>` tag with URL-encoded characters and alternative event syntax. The payload injected via the URL parameter successfully triggered an alert displaying the `document.domain` value, confirming XSS execution even under the High security filter. The High Reflected XSS Source code on the right panel shows the regex pattern used, confirming the bypass technique was effective. The injected payload is visible in the browser's address bar in encoded form.

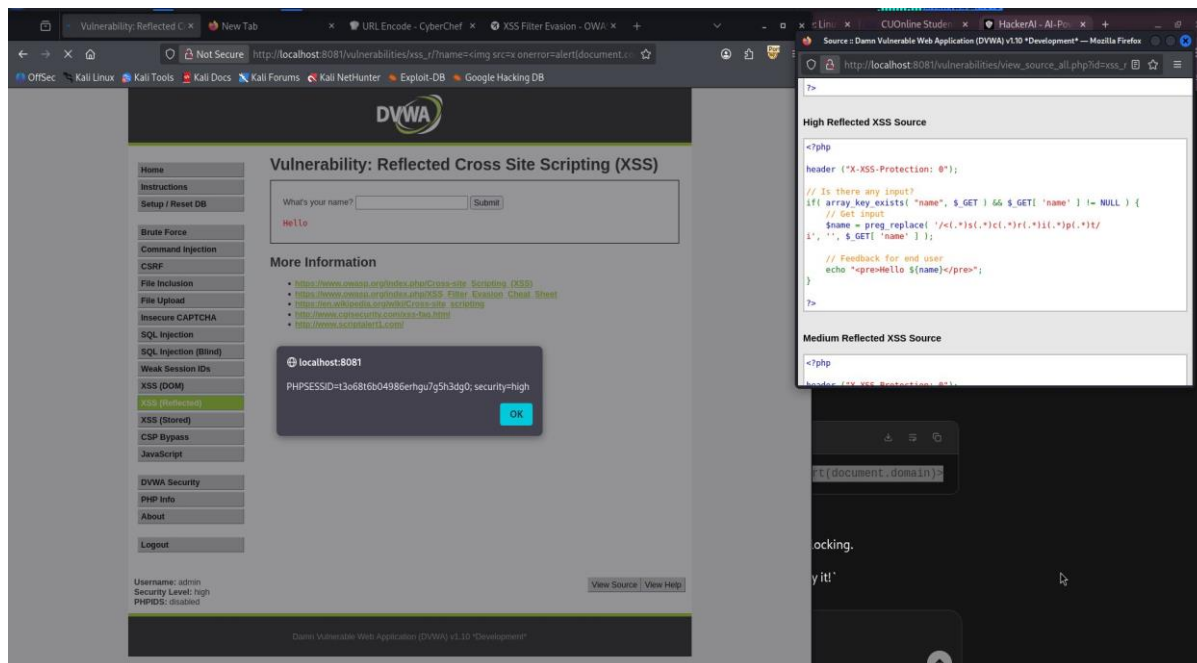


Figure – XSS High Level: DVWA Reflected XSS (High) bypassed with URL-encoded `img` tag payload. Alert box displays `document.domain`, confirming JavaScript execution. Source code shows the `preg_replace` regex filter that was evaded.